Week 04: Features & Matching

Hao Wang
AI Thrust, INFO Hub



Recap of last week



The Challenge

How Many Coins?

- You can count them by eye
- But what if there were **1,000**?
- Or **10,000**?

Goal: By the end of today, we'll make the computer count them automatically.



Sobel Edge detection

Canny Edge detection

Contour extraction

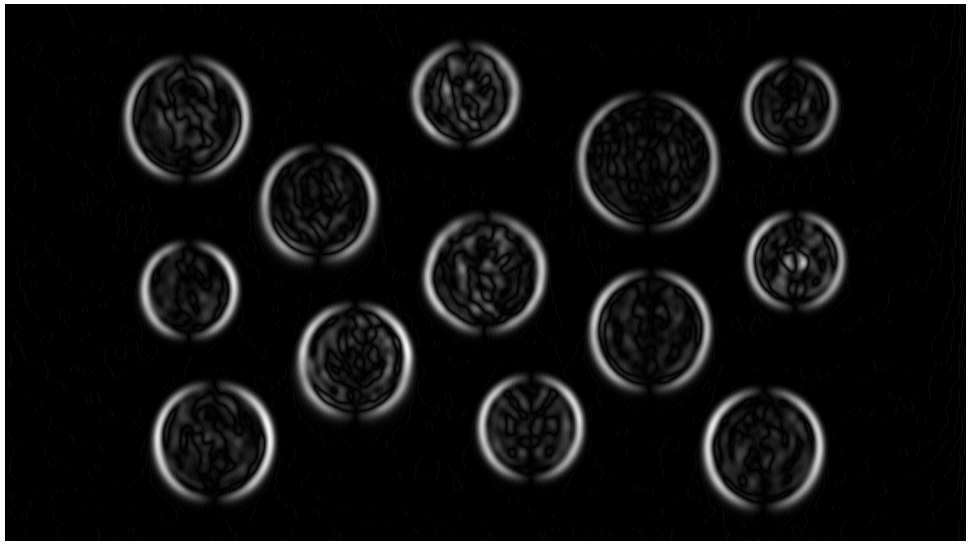
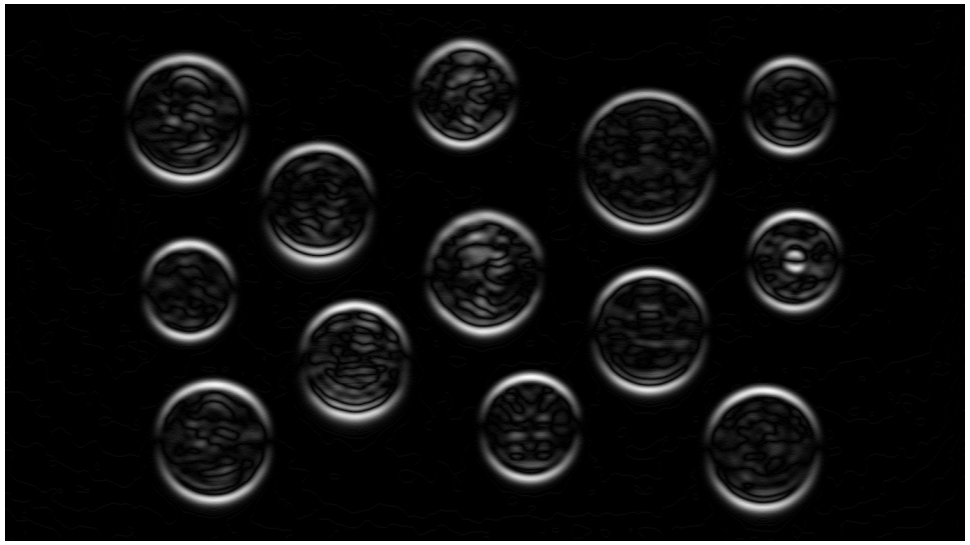


The Sobel Operator

Computing Gradients with Convolution

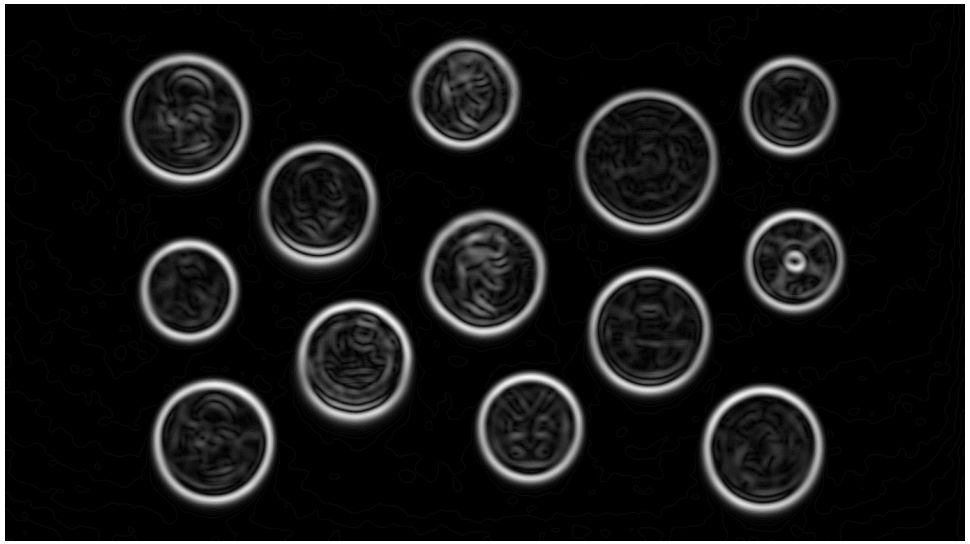


Sobel X and Y Results

Sobel X (G_x) — Vertical edges	Sobel Y (G_y) — Horizontal edges
	
Brightness changes left → right	Brightness changes top → bottom



Gradient Magnitude: All Edges Combined

Original	Gradient Magnitude $\left\|\nabla I\right\|$
	
Input image	$\sqrt{G_x^2 + G_y^2}$ — all edge directions combined

Sobel: Limitations

Issue	Description
Thick edges	Not single-pixel precision
Still noise-sensitive	Better than simple diff, but not great
No thresholding	Doesn't tell us *which* pixels are edges

We need a more complete solution...

> "Sobel gives us good gradients, but we need an entire *pipeline* to turn gradients into clean, precise edges. Enter **Canny**."



Canny Edge Detection

The Gold Standard — 40 Years and Still Counting



Five-Step Pipeline — Overview

Step	Operation	Purpose	Depth
1	Gaussian Smoothing	Remove noise	Brief (Week 2)
2	Gradient Computation	Find candidate edges	Brief (just learned Sobel)
3	**Non-Maximum Suppression**	**Thin edges to single pixel**	**Detailed**
4	Dual Threshold	Classify strong vs weak edges	Concept
5	Edge Tracking by Hysteresis	Connect valid weak edges	Concept

> Image → Smooth → Gradient → NMS → Threshold → Track → Edges



Step 3: Non-Maximum Suppression (NMS)

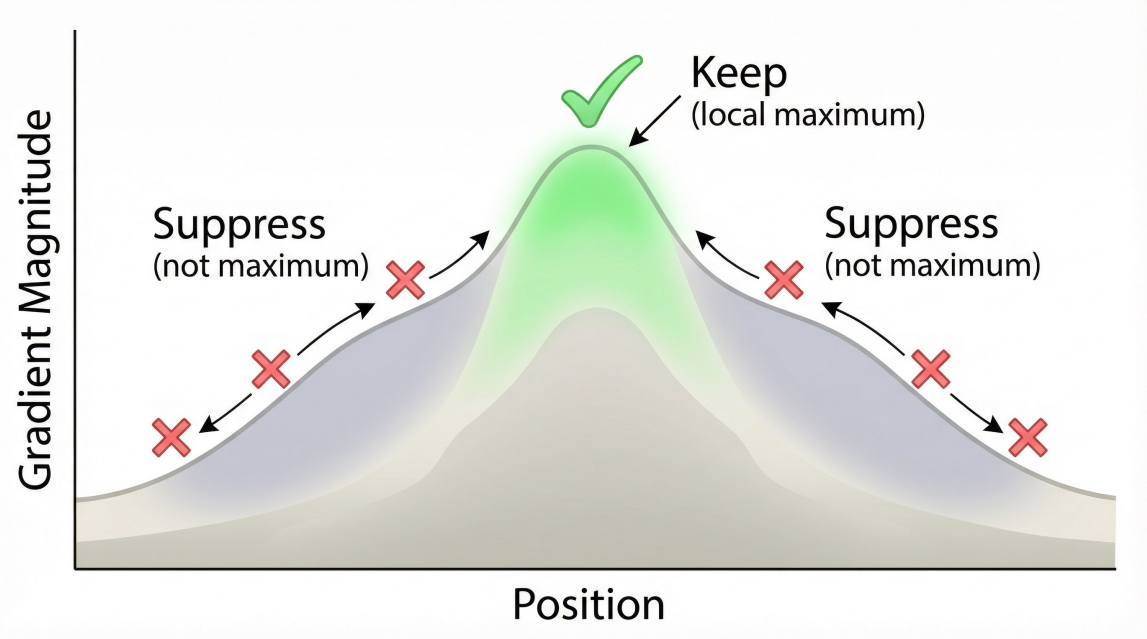
The Core Innovation of Canny

Problem: Edges are **thick bands** (5–10 pixels wide)

Goal: Keep only the **ridge** — the pixel with the largest gradient

Result: Edges become **single-pixel thin**

> Along the gradient direction, keep the **local maximum**, suppress everything else.

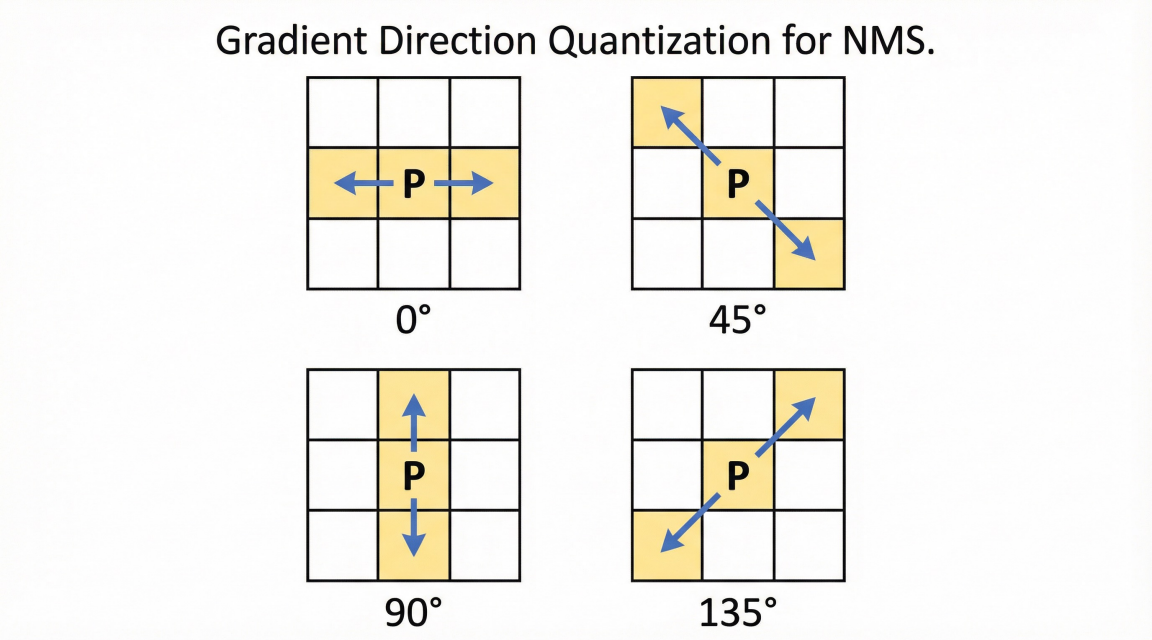


NMS: Direction Quantization

Continuous θ is quantized into **4 directions**:

Angle Range	Direction	Compare With
−22.5°–22.5°	0°	Left & Right
22.5°–67.5°	45°	↖ & ↘
67.5°–112.5°	90°	Top & Bottom
112.5°–157.5°	135°	↗ & ↙

Rule: Current ≥ both neighbors → **keep**; else → **suppress**.

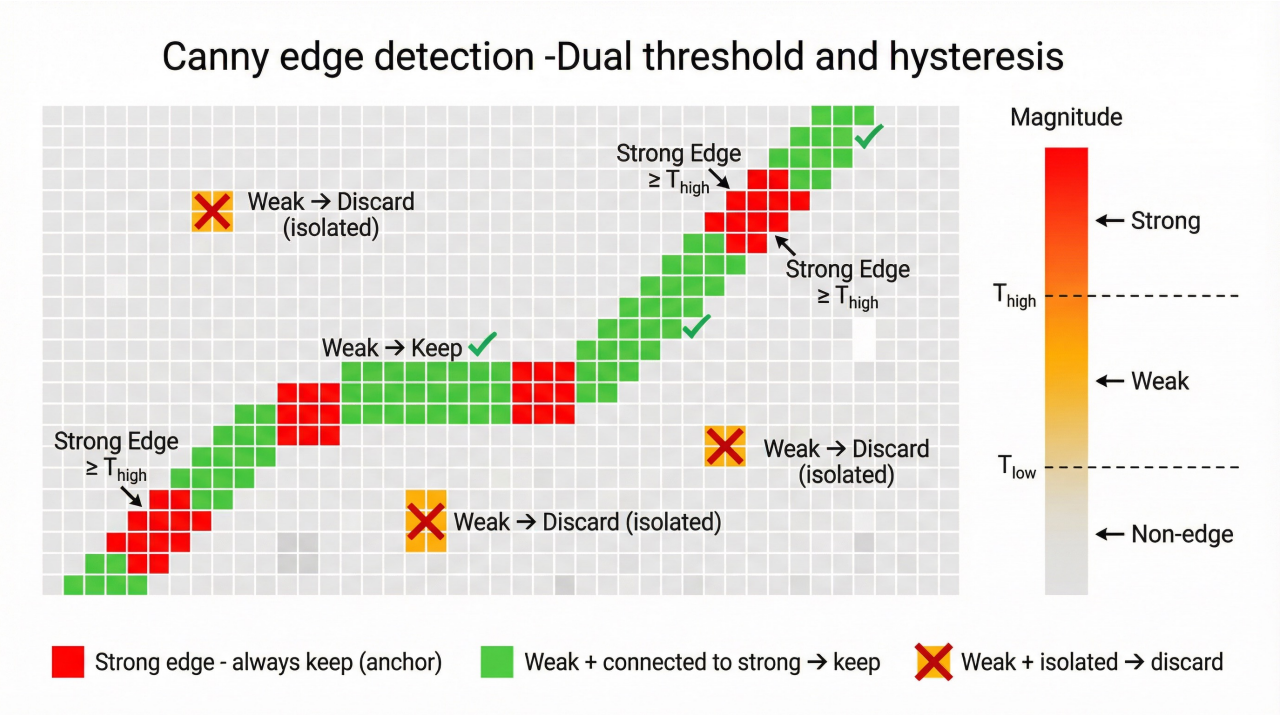


Step 4: Dual Threshold

Single threshold: an impossible trade-off

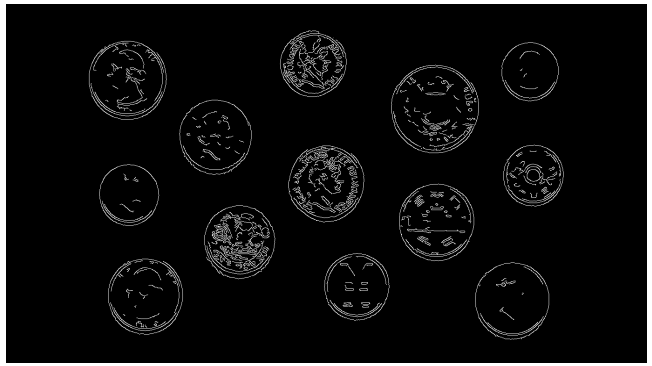
	T too high	T too low
Noise	✓ suppressed	✗ false edges
Continuity	✗ edges break	✓ complete

Solution: Use **two** thresholds.



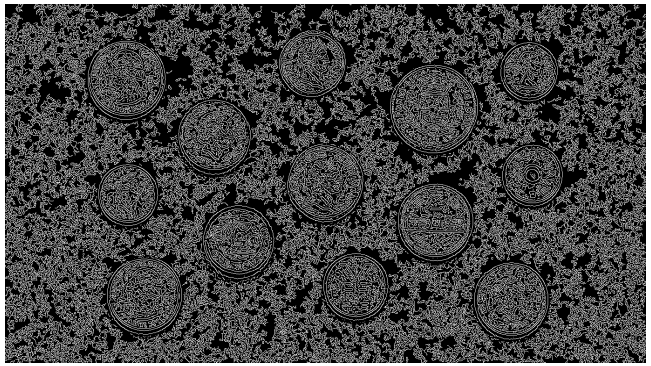
Threshold Comparison

Too High (100, 200)



Edges broken, gaps ✗

Too Low (10, 50)



Too many false edges ✗

Dual Threshold: Three Edge Classes

Classify by Strength

Class	Rule
Strong $\geq T_{high}$	Always keep — **anchor**
Weak $\in [T_{low}, T_{high})$	Keep **only if connected** to strong
Non-edge $< T_{low}$	Always discard

> **Key insight**: strength alone isn't enough — **connectivity** decides.

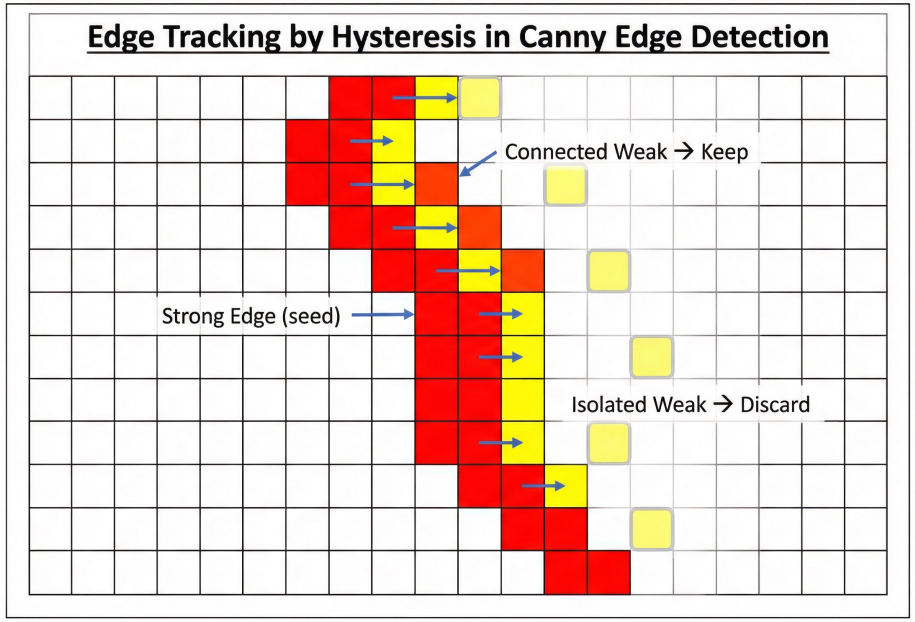


Step 5: Edge Tracking by Hysteresis

Connecting the Dots

After dual thresholding, we have **strong** and **weak** edges:

1. **Start** from each strong edge pixel (seeds)
2. **Search** 8-connected neighbors for weak edges
3. **Promote** connected weak edges → strong
4. **Discard** isolated weak edges (noise)



Strong edges are **seeds**. Weak edges near seeds **grow**. Isolated weak edges die!

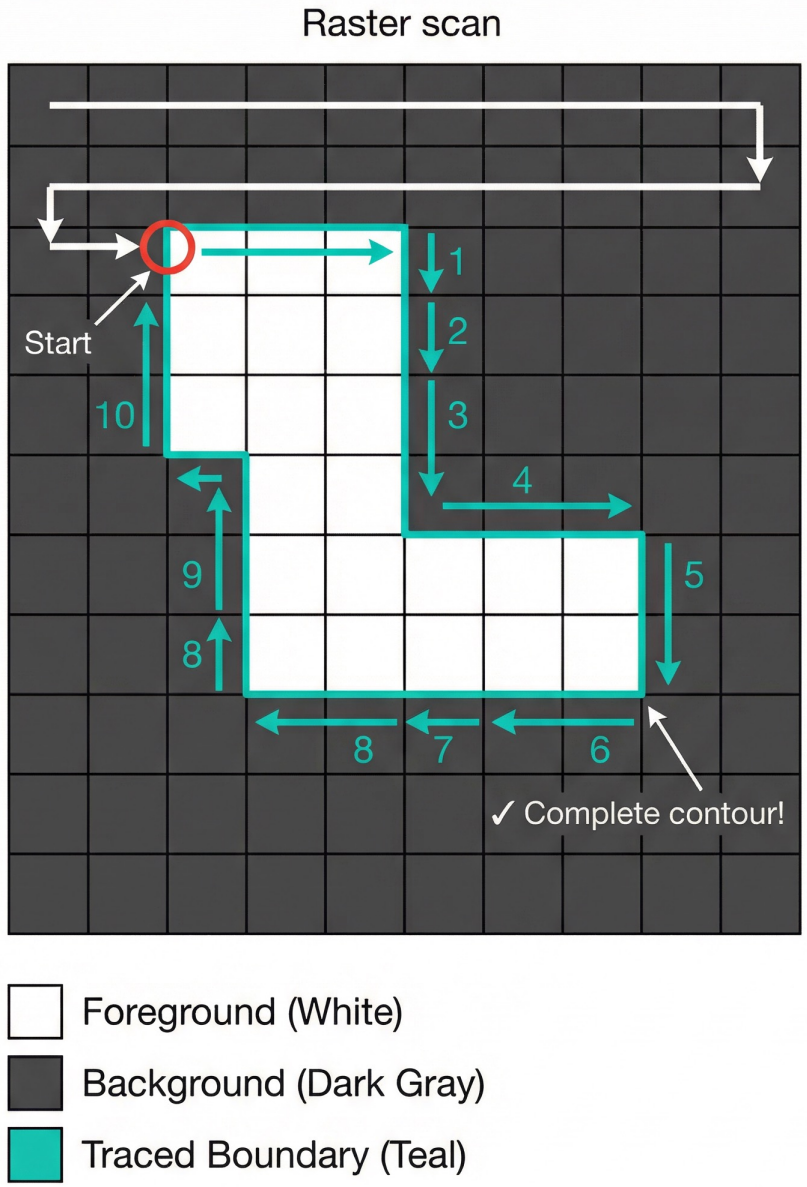
The Core Problem

Connecting Edge Pixels

Given a binary image (white foreground, black background):

1. **Find** where each boundary starts
2. **Trace** along the boundary pixel by pixel
3. **Return** to start → one complete contour
4. **Repeat** for all boundaries

"Blind Person Touching a Wall"



Opening Quiz



Quick Quiz — Q1

In Canny edge detection, why is edge tracking by hysteresis necessary after dual thresholding?

A. To further smooth the image and reduce high-frequency noise

B. To connect weak edge pixels that are likely part of the same physical edge as strong ones

C. To ensure all detected edges have single-pixel width

D. To make the algorithm invariant to rotation and scale

Answer: B

- After dual thresholding, some real edges are weak and may be broken. Hysteresis keeps weak edge pixels only if they are connected to strong edges, ensuring edge continuity while removing isolated noise.



The matching Challenge

- Your brain finds matching structures (windows, corners, signs)
- Computer vision equivalent: find **correspondences**

$$(x_i, y_i) \leftrightarrow (x_i', y_i')$$

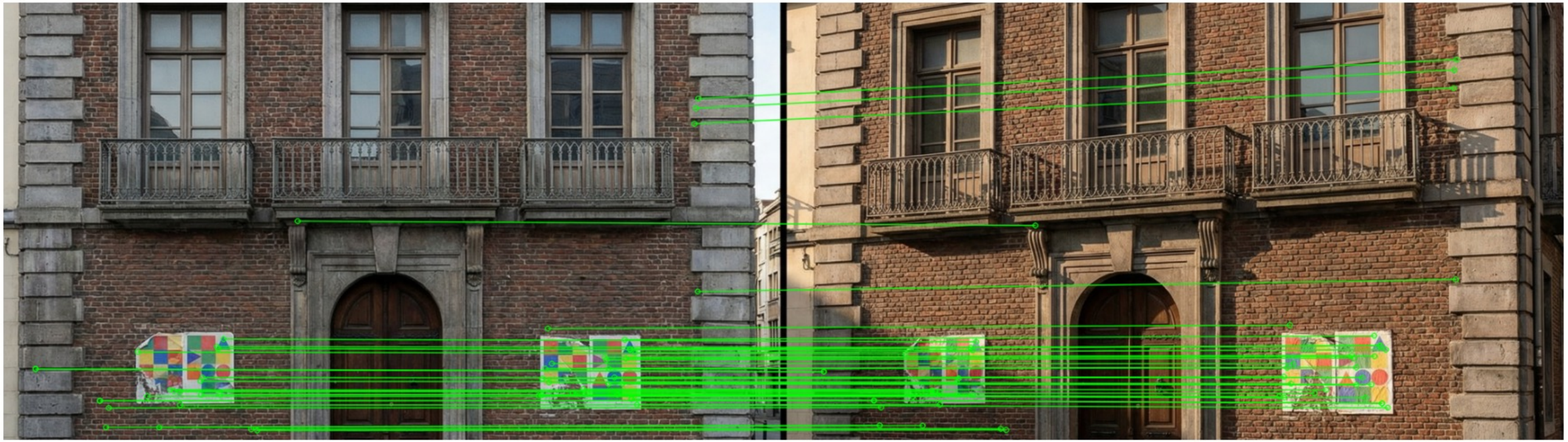
Input: Two views of the same scene.

Core Problem:
- Pixels change (Intensity, Color).
- Geometry changes (Perspective).

Goal:
Find Invariants —anchors that persist across these changes.



Finding Correspondences

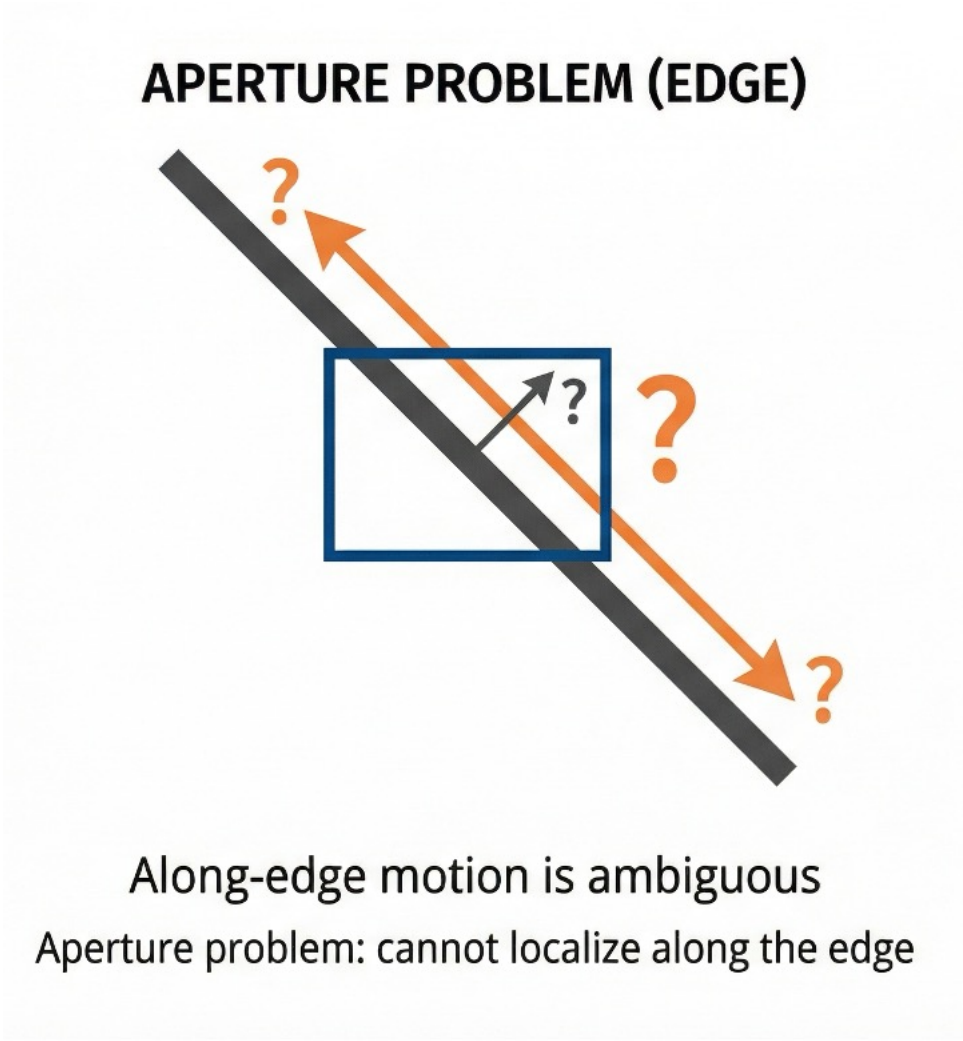


Can We Use Edge to Match Them?



Problem

- Inside a small window, you only sense change **orthogonal** to an edge
- Sliding **along** the edge: intensity barely changes
- Edge points **cannot be distinguished** from their neighbours



What Makes a Good Feature to Match two Images?

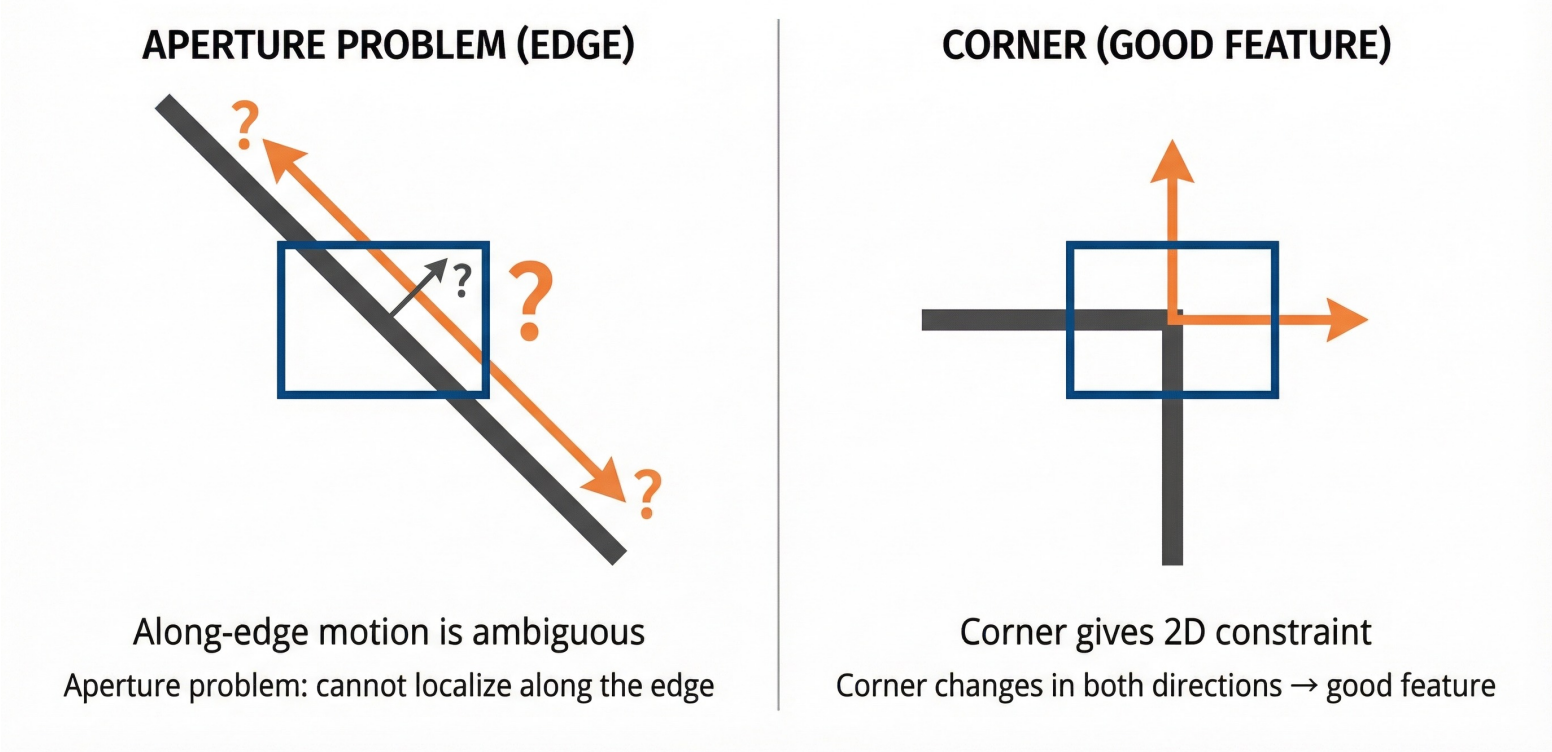


Two Hard Requirements

Criterion	Definition	Analogy
Repeatability	Detectable across viewpoints and lighting	A landmark you recognise from any direction
Distinctiveness	Won't be confused with other points	A landmark unlike all others nearby



Answer: Corner!



Why Are Corners Better Than Edges?

	Directions of change	Localisability
Flat region	0	Poor
Edge	1 direction	Can't localise along edge
Corner	**2 directions**	**Precise and unique**

- Corners have two (or more) edges meeting
- Change in **2D** → provides a full positional constraint



Corner Detection: Harris

How Do We Define a Corner, Mathematically?



From Window Shift to a Math Quantity

Slide window W around point (x, y), shift by (u, v), measure intensity change:

$$E(u,v) = \sum_{(i,j)\in W} \left[I(i+u, j+v) - I(i,j)\right]^2$$

- Small $E \rightarrow$ little change in any direction (flat)
- Large $E \rightarrow$ significant change (edge or corner)



How can we compute it in practice?



Derivation: First-Order Taylor Expansion

$$E(u,v) = \sum_{(i,j)\in W} [I(i+u, j+v) - I(i,j)]^2$$

For small (u, v), expand $I(i+u, j+v)$ around (i, j):

$$I(i+u, j+v) \approx I(i,j) + I_x\, u + I_y\, v$$

where $I_x = \frac{\partial I}{\partial x}$, $I_y = \frac{\partial I}{\partial y}$ (image gradients)

Intensity difference:

$$I(i+u, j+v) - I(i,j) \approx I_x\, u + I_y\, v$$

$$(I_x u + I_y v)^2 = I_x^2 u^2 + 2 I_x I_y uv + I_y^2 v^2$$

Substituting into $E(u, v)$ and rearranging into matrix form:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\sum_W \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}}_{M} \begin{bmatrix} u \\ v \end{bmatrix}$$

M = **Structure Tensor** — captures **gradient energy and correlation** within the window

Orthogonal diagonalization: $M = Q\Lambda Q^\top$, with $\Lambda = \mathrm{diag}(\lambda_1, \lambda_2)$ and columns of Q the two eigen-directions.

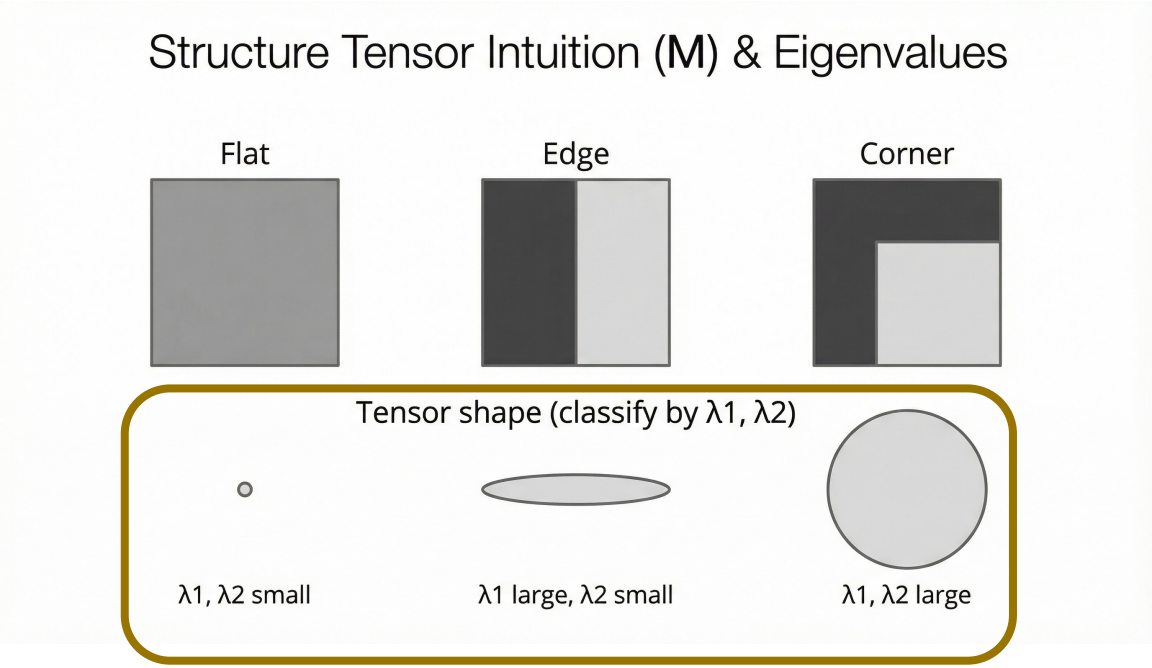
Let $\begin{bmatrix} u' \\ v' \end{bmatrix} = Q^\top \begin{bmatrix} u \\ v \end{bmatrix}$ (displacement in eigen-coordinates). Then:

$$E(u,v) = \begin{bmatrix} u & v \end{bmatrix} Q\Lambda Q^\top \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u' & v' \end{bmatrix} \Lambda \begin{bmatrix} u' \\ v' \end{bmatrix} = \lambda_1 (u')^2 + \lambda_2 (v')^2$$

Structure Tensor Intuition

$$M = \sum_W \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- **Flat**: $I_x \approx I_y \approx 0 \rightarrow M \approx 0$
- **Edge**: large gradient in one direction only, when crossing the boundary
- **Corner**: large gradients in both directions



So:

- Along eigen-direction 1, intensity change is governed by λ_1
- Along eigen-direction 2, by λ_2

Eigenvalue-Based Classification

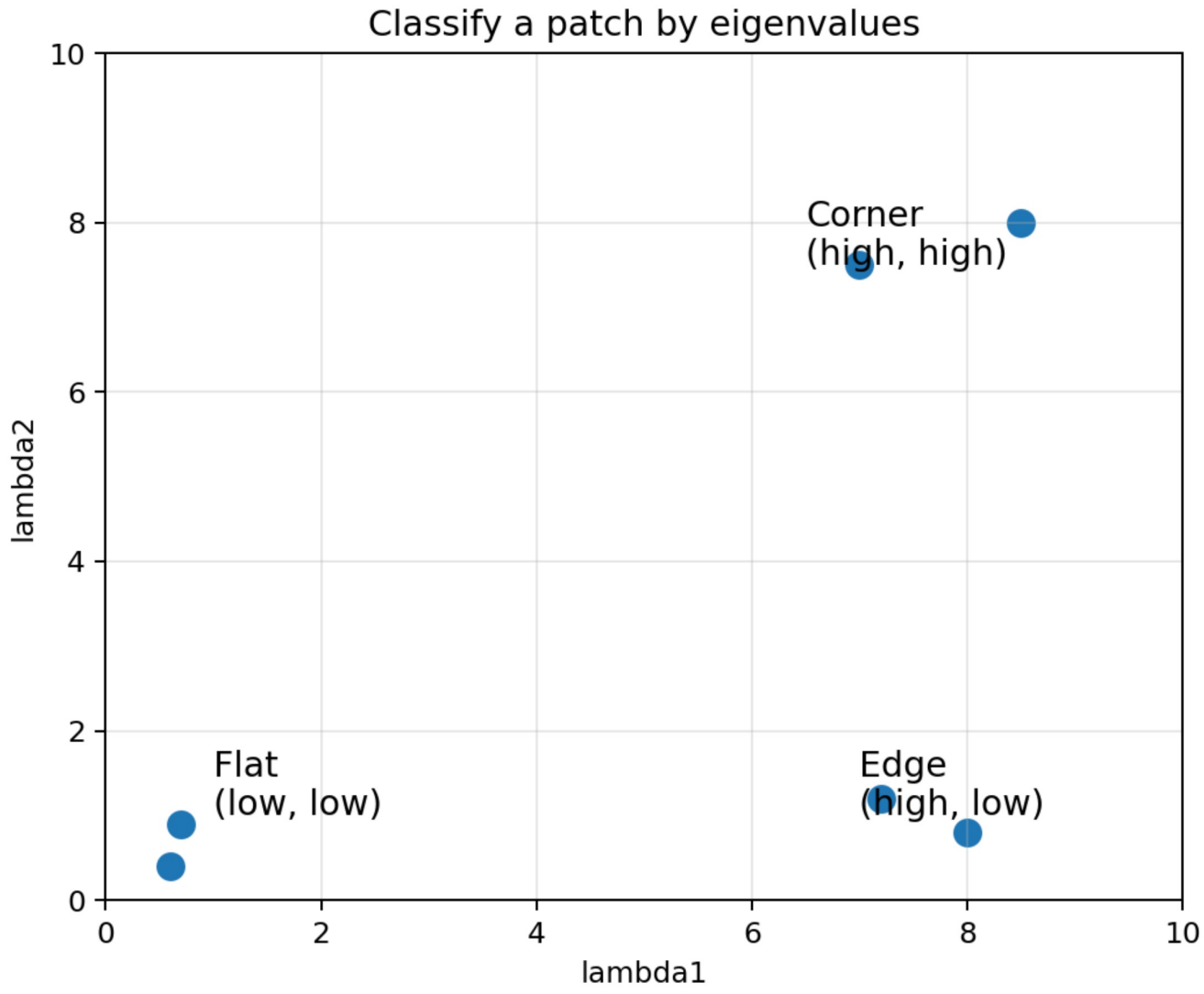
Eigenvalues $\lambda_1 \geq \lambda_2 \geq 0$ of M represent change magnitudes along principal axes:

Case	λ_1	λ_2	**Meaning**
Flat	small	small	Intensity barely changes in any direction
Edge	large	small	Change in one direction only
Corner	large	large	Change in both directions



Eigenvalue Plane: Visual Classification

- **Flat**: both λ_1, λ_2 small
- **Edge**: λ_1 large, $\lambda_2 \approx 0$
- **Corner**: both λ_1, λ_2 large



Problem: solving the characteristic equation explicitly is **expensive**!

Harris Response Function R

Key insight: $\det(M)$ and $\mathrm{trace}(M)$ equal the product and sum of eigenvalues:

$$\det(M) = \lambda_1 \lambda_2, \qquad \mathrm{trace}(M) = \lambda_1 + \lambda_2$$

Harris response:

$$R = \det(M) - k \cdot \mathrm{trace}(M)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

No eigenvalue solving needed — compute directly from matrix elements!



Proof: det = $\lambda_1\lambda_2$, trace = $\lambda_1 + \lambda_2$

$$M = \begin{bmatrix} a & b \\ b & c \end{bmatrix} = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

$$\det(M - \lambda I) = 0$$

$$\det(M - \lambda I) = (a - \lambda)(c - \lambda) - b^2$$

$$\lambda^2 - (a + c)\lambda + (ac - b^2) = 0$$

For a quadratic equation

$$x^2 - Sx + P = 0$$

By **Vieta's formulas**:

$$x_1 + x_2 = S$$

$$x_1 x_2 = P$$

$$\lambda^2 - (\lambda_1 + \lambda_2)\lambda + \lambda_1\lambda_2 = 0$$



Proof: det = $\lambda_1\lambda_2$, trace = $\lambda_1 + \lambda_2$

The same characteristic polynomial $\det(M - \lambda I) = 0$ written two ways:

$$\lambda^2 - \underbrace{(a+c)}_{\text{trace}(M)}\lambda + \underbrace{(ac - b^2)}_{\det(M)} = 0$$

Expansion	λ^1 **coefficient**	λ^0 **coefficient**
From matrix	$-(a+c) = -\text{trace}(M)$	$ac - b^2 = \det(M)$
From roots	$-(\lambda_1 + \lambda_2)$	$\lambda_1\lambda_2$

Matching coefficients $\rightarrow \text{trace}(M) = \lambda_1 + \lambda_2$, $\det(M) = \lambda_1\lambda_2$ ✓



Computing det and trace: Extremely Cheap

Let $a = \sum_W I_x^2,\ b = \sum_W I_x I_y,\ c = \sum_W I_y^2$

$$\text{trace}(M) = a + c \qquad \det(M) = ac - b^2$$

Full computation chain:

$$\text{Image} \xrightarrow{\text{Sobel}} I_x, I_y \xrightarrow{\text{window sum}} a, b, c \xrightarrow{\det,\ \text{trace}} R$$



How R Classifies: Flat / Edge / Corner

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

$$R = \det(M) - k \cdot \mathrm{trace}(M)^2$$

$$\det(M) = \lambda_1 \times \lambda_2 = I_x^2 I_y^2 - (I_x I_y)^2 \qquad \mathrm{trace}(M) = \lambda_1 + \lambda_2 = I_x^2 + I_y^2$$

- **Corner** (both λ large): $R \gg 0$
- **Flat** (both small): $R \approx 0$
- **Edge** ($\lambda_2 \approx 0$): $R \approx -k\lambda_1^2 < 0$

Typical k value: $0.04 \sim 0.06$



Engineering Post-Processing

NMS + Threshold + Spatial Uniformity



Non-Maximum Suppression (NMS)

Keep only the pixel with the highest R in each local neighbourhood:

```
Raw R map:       After NMS:
  2  5  3           0  0  0
  4  8  6    →      0  8  0
  1  3  2           0  0  0
(only 8 is the local maximum in this 3×3 patch)
```

- Intuition: R response is a "peak" — NMS keeps only the summit
- Result: each real corner is represented by **exactly one pixel**



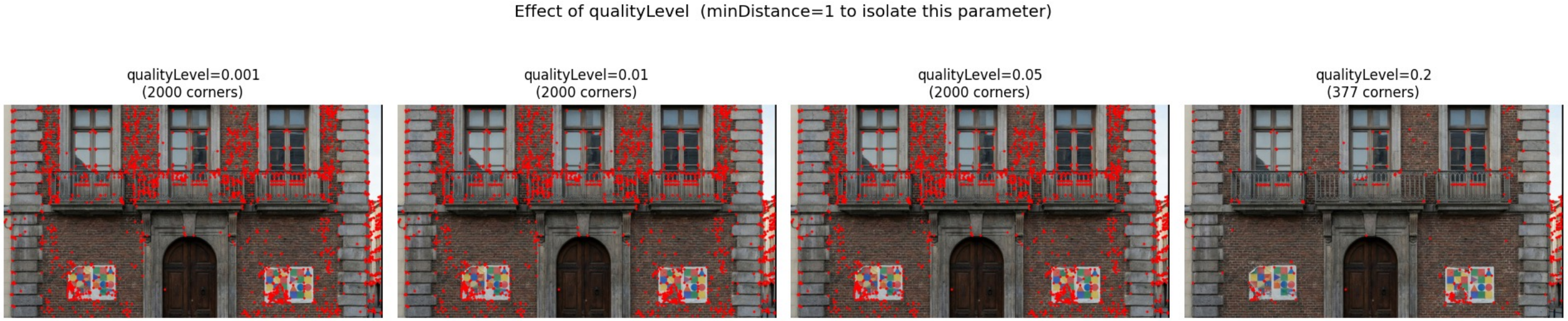
Non-Maximum Suppression (NMS)





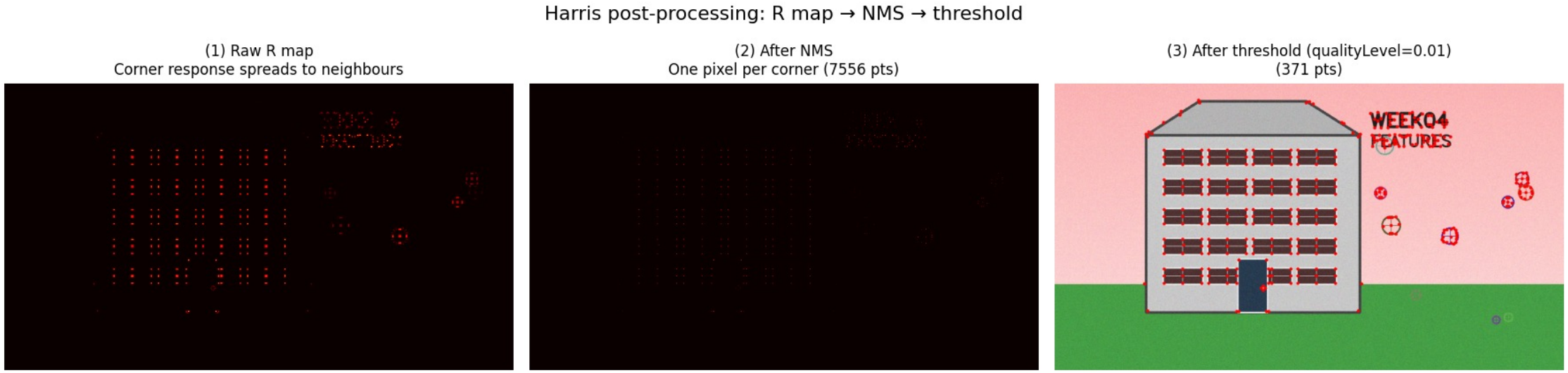
Thresholding

Keep only $R > R_{max} * qualityLevel$





NMS Effect

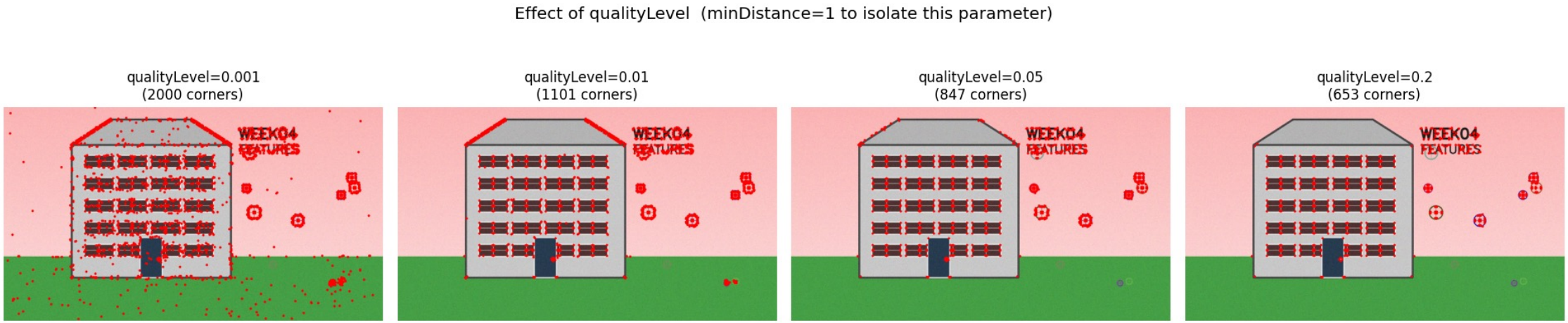


Left: raw R map (response spreads over many pixels); Centre: after NMS (one pixel per corner); Right: overlaid on original image



Threshold / top-K Filtering

Keep only $R > R_{max} * qualityLevel$

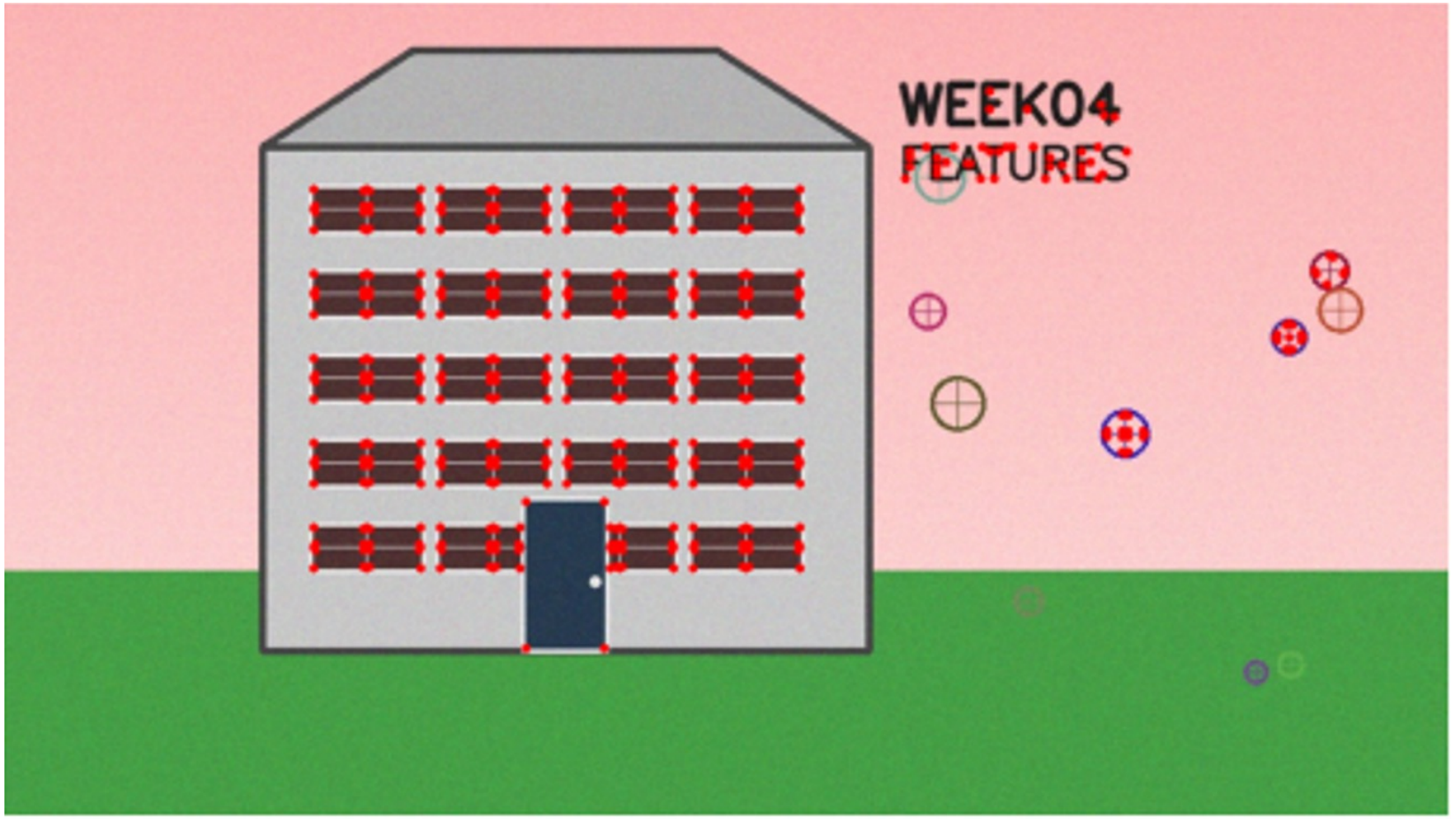


`qualityLevel` increasing left to right (0.001 → 0.2) → corner count



Minimum Distance Constraint: `minDistance`

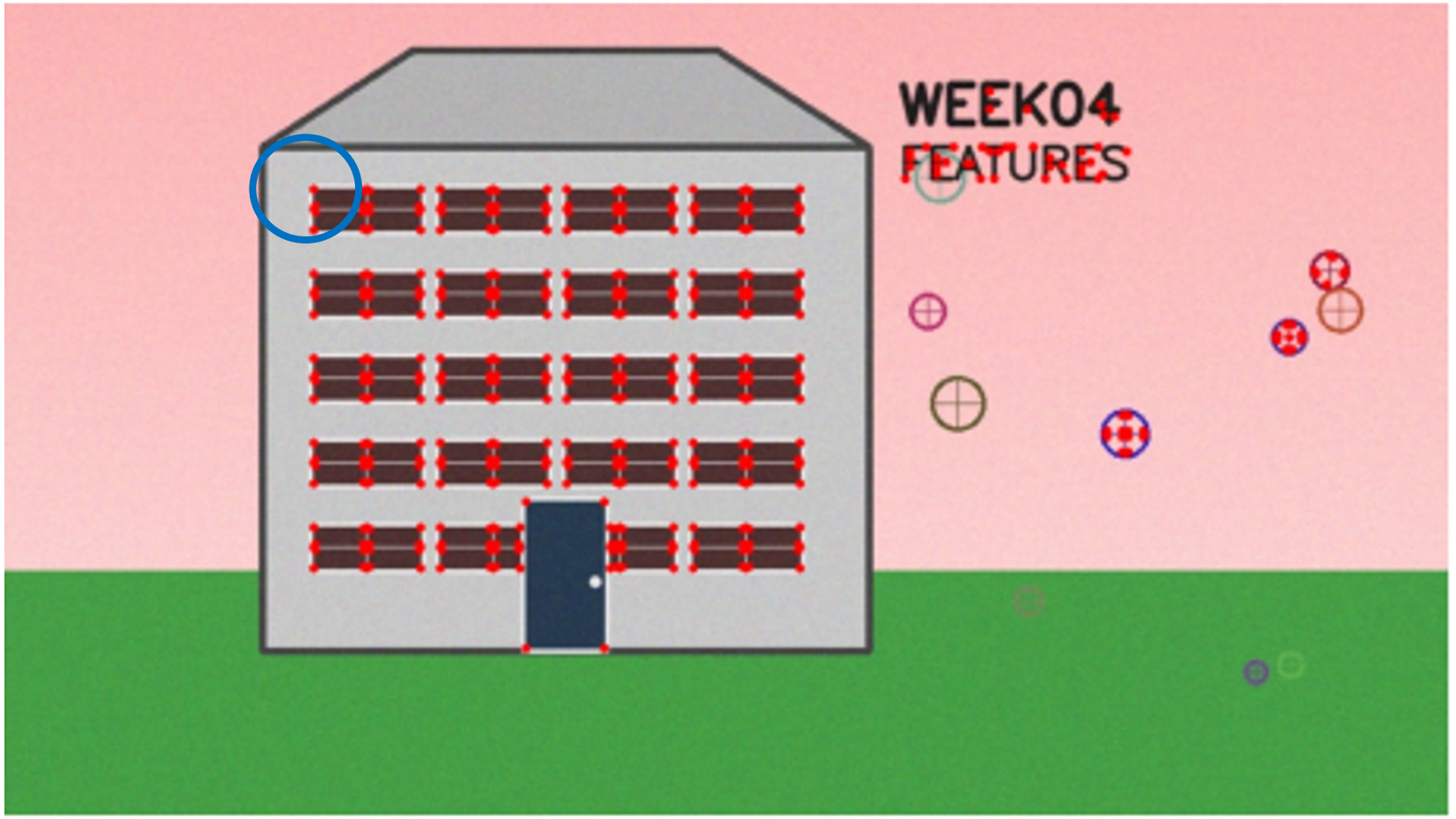
Sort R value from large to small



Minimum Distance Constraint: `minDistance`

Sort R value from large to small

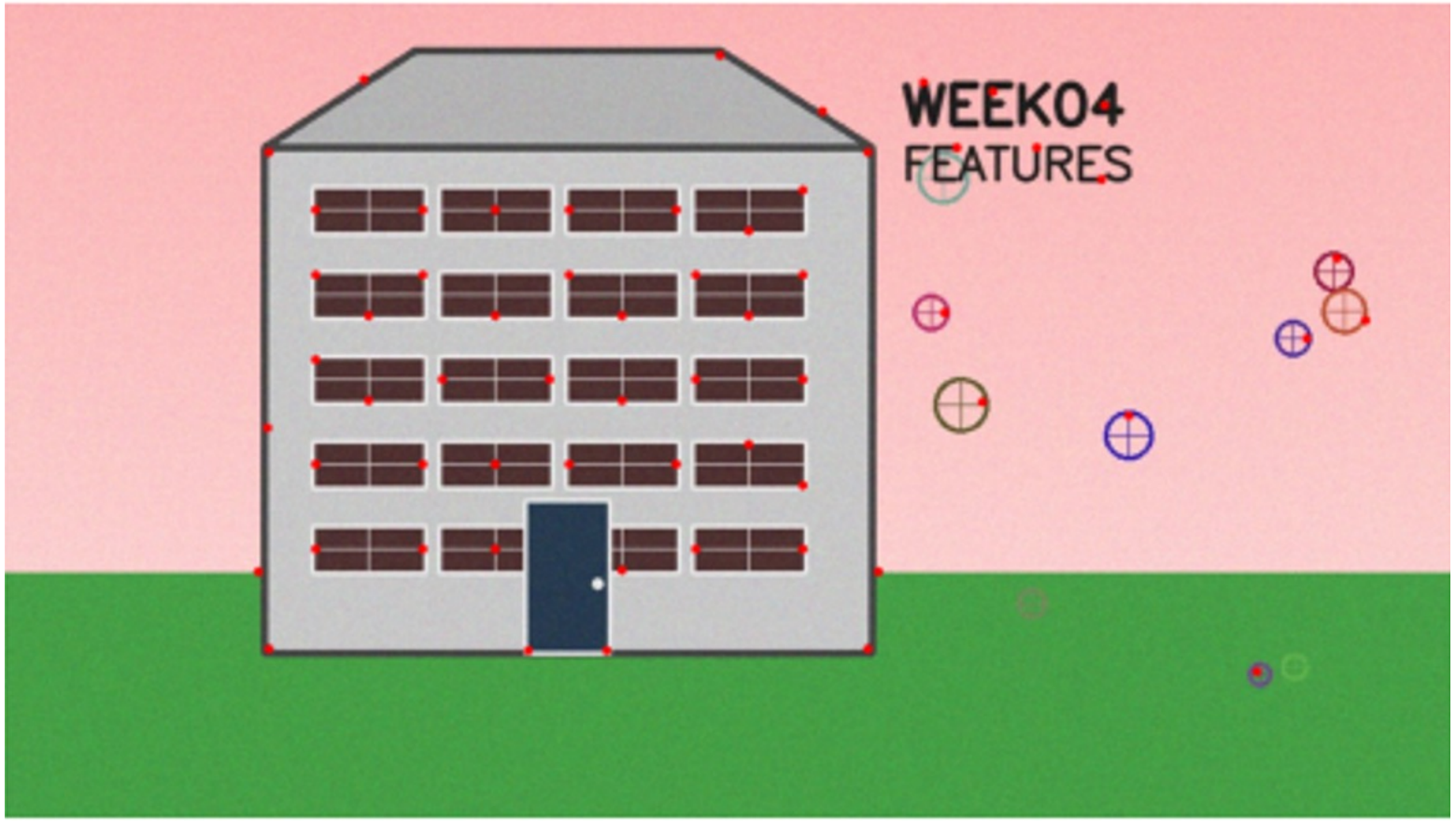
For R_i from Largest to Smallest:
Set its neighbors within minDistance to 0



Minimum Distance Constraint: `minDistance`

Sort R value from large to small

For R_i from Largest to Smallest:
Set its neighbors within minDistance to 0



Minimum Distance Constraint: `minDistance`



Greedily select by R value, excluding candidates within `minDistance` radius → evenly distributed corners

Minimum Distance Constraint:

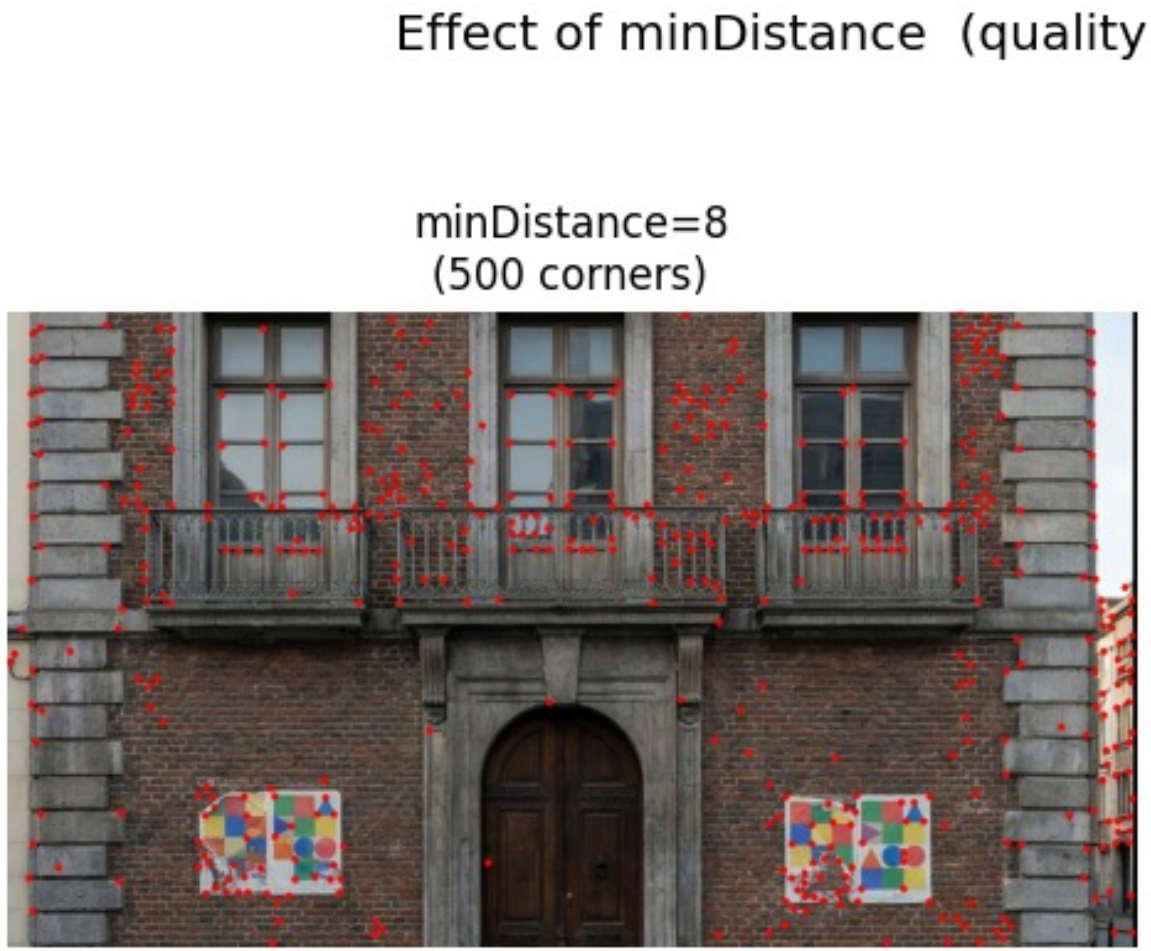
minDistance=1
(500 corners)



Preventing Cluster Overload

- **The Problem**: Features often cluster in high-texture areas, leaving the rest of the image "blind."
- **The Solution**:
  - Sort candidates by R.
  - Pick the strongest, then **suppress** all others within a radius r.
- **Result**: Evenly distributed anchors.

Minimum Distance Constraint:



Preventing Cluster Overload

- **The Problem**: Features often cluster in high-texture areas, leaving the rest of the image "blind."
- **The Solution**:
  - Sort candidates by R.
  - Pick the strongest, then **suppress** all others within a radius r.
- **Result**: Evenly distributed anchors.

Minimum Distance Constraint:

minDistance=1
(500 corners)



Preventing Cluster Overload

- **The Problem**: Features often cluster in high-texture areas, leaving the rest of the image "blind."
- **The Solution**:
  - Sort candidates by R.
  - Pick the strongest, then **suppress** all others within a radius r.
- **Result**: Evenly distributed anchors.

Minimum Distance Constraint:

Level=0.01, maxCorners=500)

minDistance=20
(500 corners)



Preventing Cluster Overload

- **The Problem**: Features often cluster in high-texture areas, leaving the rest of the image "blind."
- **The Solution**:
  - Sort candidates by R.
  - Pick the strongest, then **suppress** all others within a radius r.
- **Result**: Evenly distributed anchors.

Minimum Distance Constraint:

minDistance=40
(211 corners)



Preventing Cluster Overload

- **The Problem**: Features often cluster in high-texture areas, leaving the rest of the image "blind."
- **The Solution**:
  - Sort candidates by R.
  - Pick the strongest, then **suppress** all others within a radius r.
- **Result**: Evenly distributed anchors.

Harris: Complete Pipeline

$$\text{Image} \xrightarrow{\text{Sobel}} I_x, I_y \xrightarrow{\text{window sum}} M \xrightarrow{\text{det, trace}} R \text{ map}$$

$$R \text{ map} \xrightarrow{\text{NMS}} \text{local maxima} \xrightarrow{\text{threshold}} \text{candidates} \xrightarrow{\text{minDist}} \text{final corners}$$



OpenCV API

```
# Raw Harris response map
cv2.cornerHarris(gray, blockSize, ksize, k)

# Recommended: returns corner coordinates
cv2.goodFeaturesToTrack(
    gray, maxCorners,
    qualityLevel, minDistance)
```

- `qualityLevel=0.01`
- `minDistance=8–15`



Descriptors

Position alone is not enough — we need to **describe**



Keypoint vs Descriptor

Concept	Contains	Analogy
Keypoint	Location + scale + orientation + response	Home address
Descriptor	Vector describing the patch around the keypoint	Fingerprint

- Keypoint tells you **where**
- Descriptor tells you **what it looks like**



Harris Has Three Fatal Gaps

Fixed window can see the whole building from far away, while only see the brick when very near

Gap	Symptom	SIFT Solution
No scale	Near/far views yield different patch content	DoG scale space; keypoint carries σ
No orientation	Rotated views misalign patches	Dominant orientation; rotate-align before extracting
No descriptor	Position only — can't verify if same 3D point	128-D HOG vector



SIFT

Scale-Invariant Feature Transform

Harris detects keypoints, SIFT describes how the keypoint looks.



Keypoint vs Descriptor

Where vs. What

Concept	Contains	Analogy
Keypoint	(x, y) + scale (σ) + orientation (θ)	Home address
Descriptor	Vector (128-D) describing the patch	Fingerprint

- Keypoint tells you **where** the anchor is.
- Descriptor tells you **what** the neighborhood looks like.



SIFT: Three Problems, Three Solutions

1. **DoG scale space** → scale invariance
2. **Dominant orientation alignment** → rotation invariance
3. **Gradient orientation histogram** → lighting invariance + 128-D descriptor



Problem 1: Scale Invariance

Question: the same corner looks different at different distances — how do we find keypoints that are stable across scales?

→ Answer: **Difference of Gaussians (DoG) scale space**

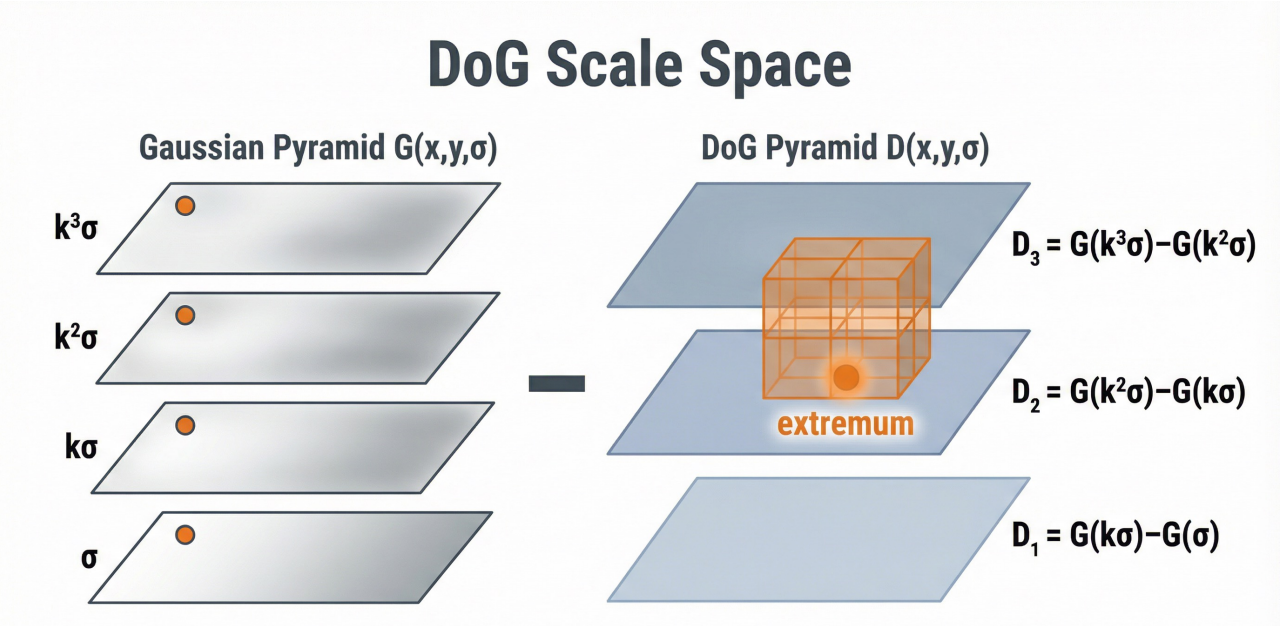


Scale Space: DoG

Build a **Gaussian pyramid** — blur at increasing σ, then subtract adjacent layers:

$$D(x, y, \sigma) = G(x, y, k\sigma) - G(x, y, \sigma)$$

- Find **3-D extrema** (x, y, scale) in the DoG volume — must be extreme among all **26 neighbours**
- Each keypoint records its **σ** → descriptor patch size σ
- Same corner at different distances → matched at its "natural" scale



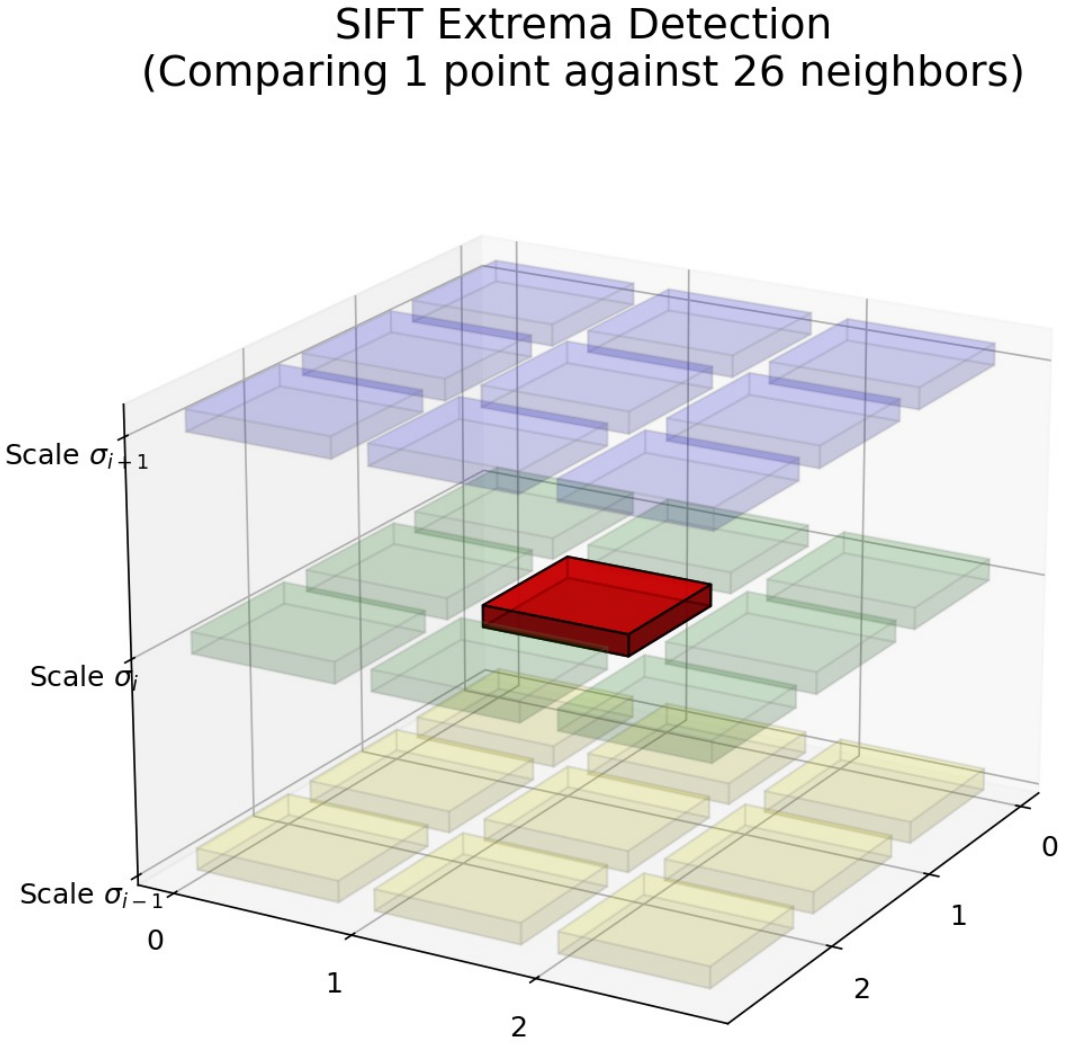
Neighbours from the same layer (8), previous (9) and next (9) layers

SIFT: Multi-scale Extrema

- **Scale Selection**: Maxima in **3D Scale Space**.

- **26-Neighbor Check**:
  - Current layer: 8.
  - Above/Below layers: 9 each.
- **Physics**: Extrema $\rightarrow$ "**blob**" matches kernel size at that σ.

Blob is more robust than the corner from Harris dection



Lowe's Keypoint Sieve

Removing the "Noise" and "Edges"

Initial detection finds too many points (~90% are junk). Lowe filters them:

1. **Low Contrast**: If $|D(\hat{x})| < 0.03$ (for 8-bit images), it's too weak $\rightarrow$ likely noise.
2. **Edge Response (Hessian Filter)**:
   - Edges are DoG maxima but unstable for localization (Aperture problem).
   - We check the **Ratio of Principal Curvatures** using the Hessian H:

$$\frac{\text{tr}(H)^2}{\det(H)} < \frac{(r+1)^2}{r}$$

   - Usually $r = 10$. If the ratio is high, one curvature is much larger than the other (an edge), so we discard it.
   - Note: This is the exact same logic as Harris R, just written as a ratio.



Problem 2: Rotation Invariance

Question: if the camera rotates, the patch around a keypoint rotates too — how do we make the descriptor independent of orientation?

→ Answer: **align to the dominant gradient orientation before describing**

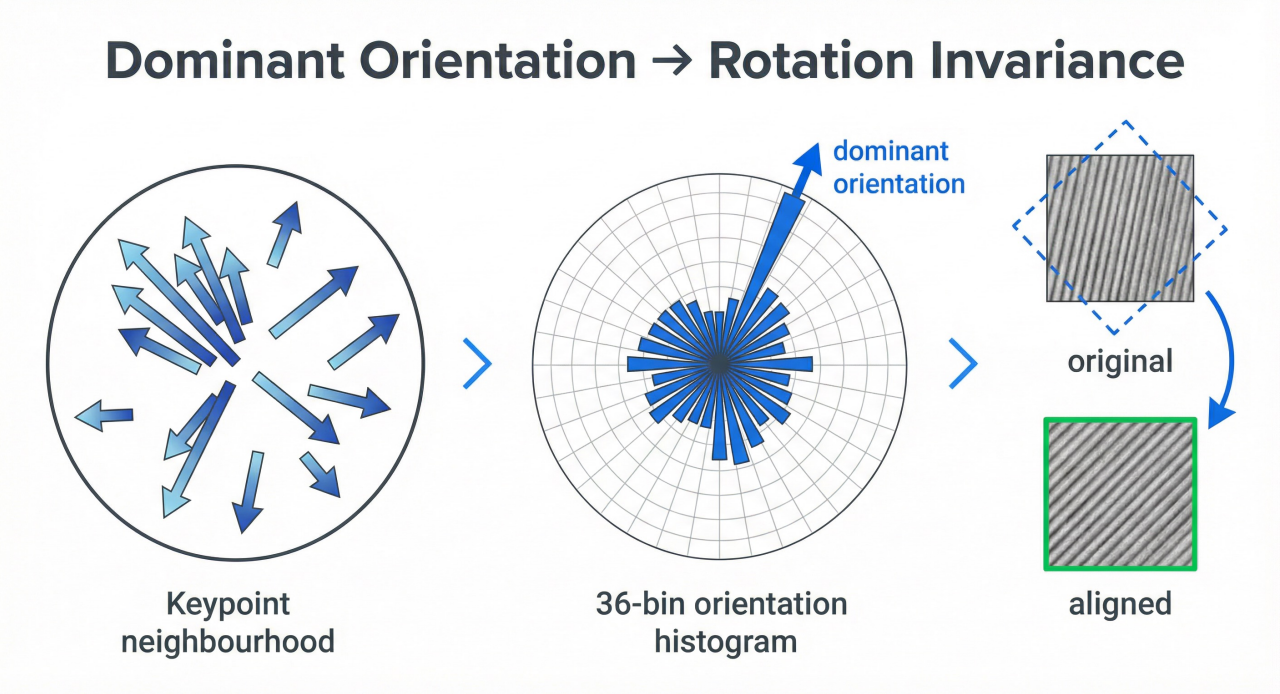


Dominant Orientation → Rotation Invariance

Step 1 — Compute a **36-bin** (10° each) gradient histogram over a large circular neighbourhood

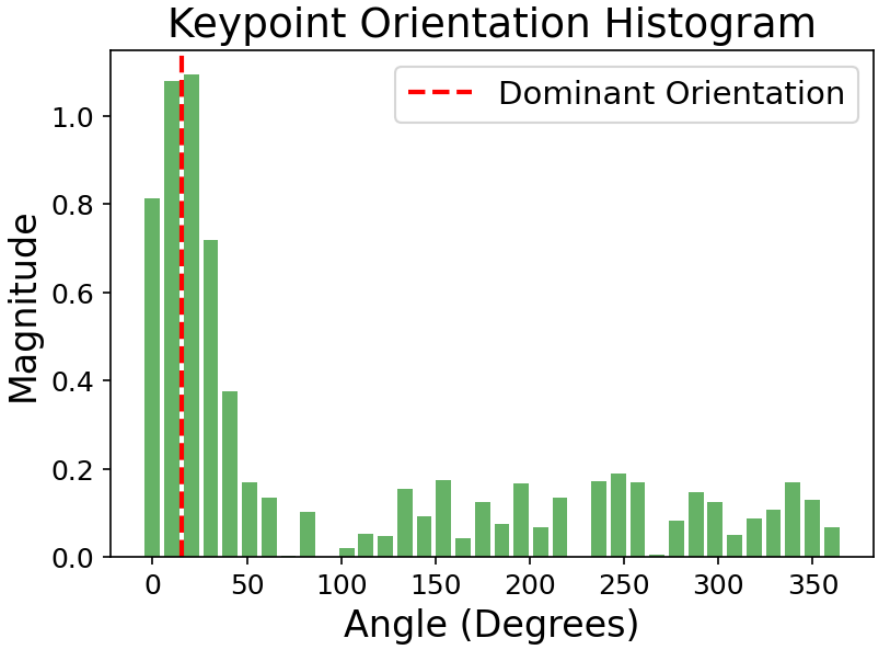
Step 2 — Take the **peak bin direction** as the keypoint's dominant orientation

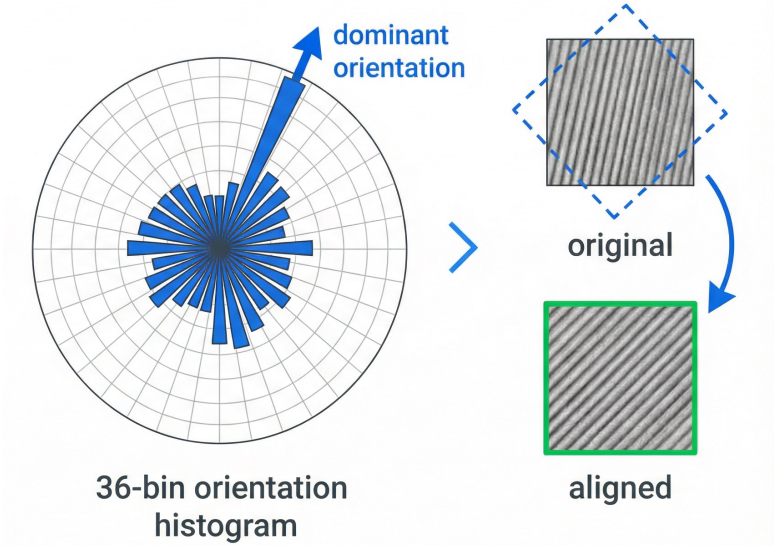
Step 3 — Rotate the coordinate frame to align with that direction before extracting the descriptor



SIFT: Rotation Invariance

- **Dominant Orientation**:
  - Build a histogram of all gradients.
  - Highest peak is the **Reference Orientation**.
- **Normalization**:
  - Rotate the coordinate system to match this peak.
  - **Result**: "Up" is always relative to the feature's own structure.





Problem 3: Lighting Invariance

Question: now that we have a scale-normalised, orientation-aligned patch — what do we put in the descriptor?

Raw pixels fail under lighting change. We need a representation that is stable to brightness shifts.

→ Answer: **gradient orientation histograms**



Why Not Raw Pixels?

Approach	Problem
Raw pixel values	Change completely under lighting variation
Pixel differences	Still sensitive to absolute brightness
Gradient directions	Edge direction is stable across lighting changes

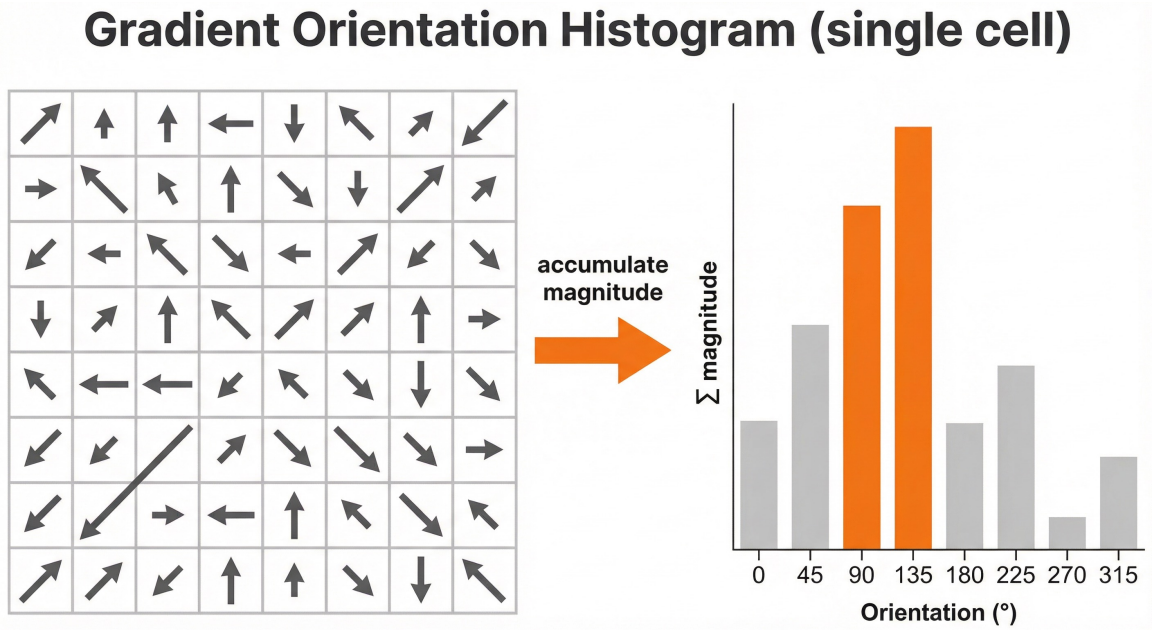


Gradient Orientation Histogram (simplified)

Per pixel in the patch:

$$\theta = \operatorname{atan}\ (I_y,\, I_x), \quad m = \sqrt{I_x^2 + I_y^2}$$

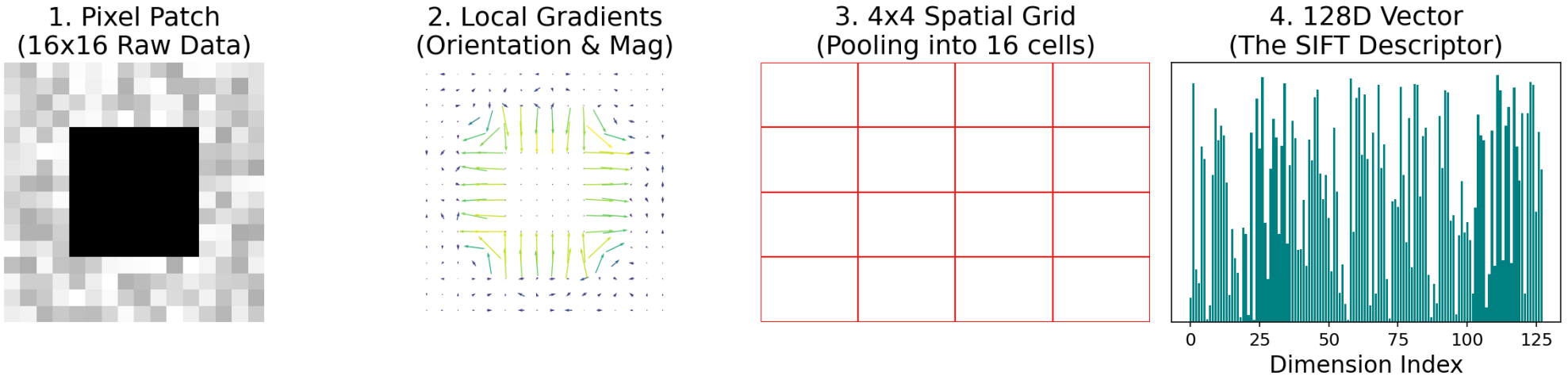
- Quantize θ into **8 bins** (45° each)
- Accumulate magnitude m into the corresponding bin
- Result: a compact, lighting-robust signature of local texture



128-D Descriptor

Orientation-aligned, scale-normalised **16×16 neighbourhood**:

1. Divide into **4×4 = 16 cells** (each 4×4 px)
2. Per cell: gradient orientation histogram, **8 bins** (45° each)
3. Concatenate → 16 × 8 = **128-D vector**
4. L2-normalise → clip at 0.2 → re-normalise → lighting robust



Feature Matching

Brute Force → Ratio Test → RANSAC



Brute-Force Matching

The $O(NM)$ Search

Match every point in A (N) to nearest in B (M).

- **Complexity**: $O(NM)$ (10^6 ops for 1k pts).
- **Scalability Wall**: 100 queries $\times 10k$ images $\rightarrow 10^9$ comparisons!
- **Need**: Mobile AR (30 FPS) requires sub-linear retrieval.



Brute-Force Matching



For every descriptor, find the nearest neighbour in the other image → many false matches

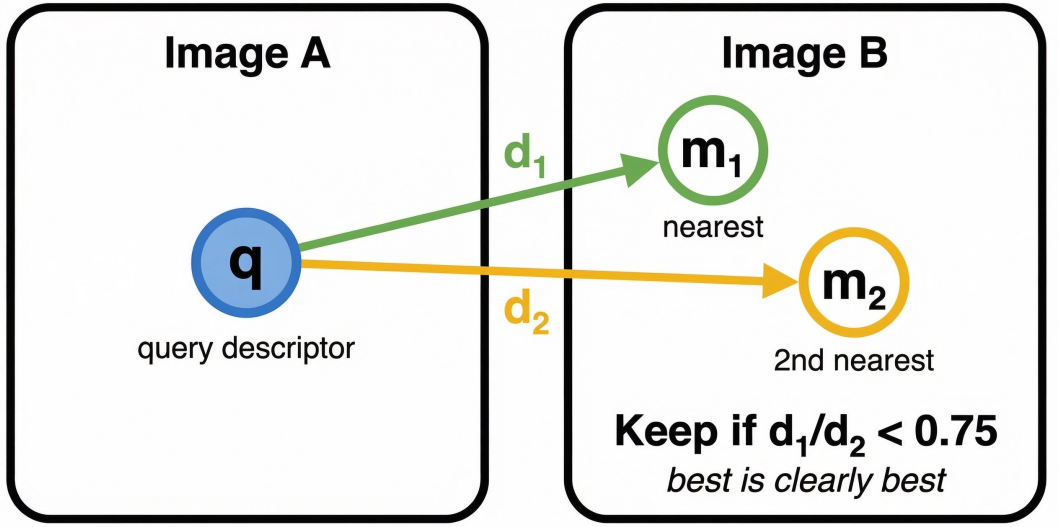


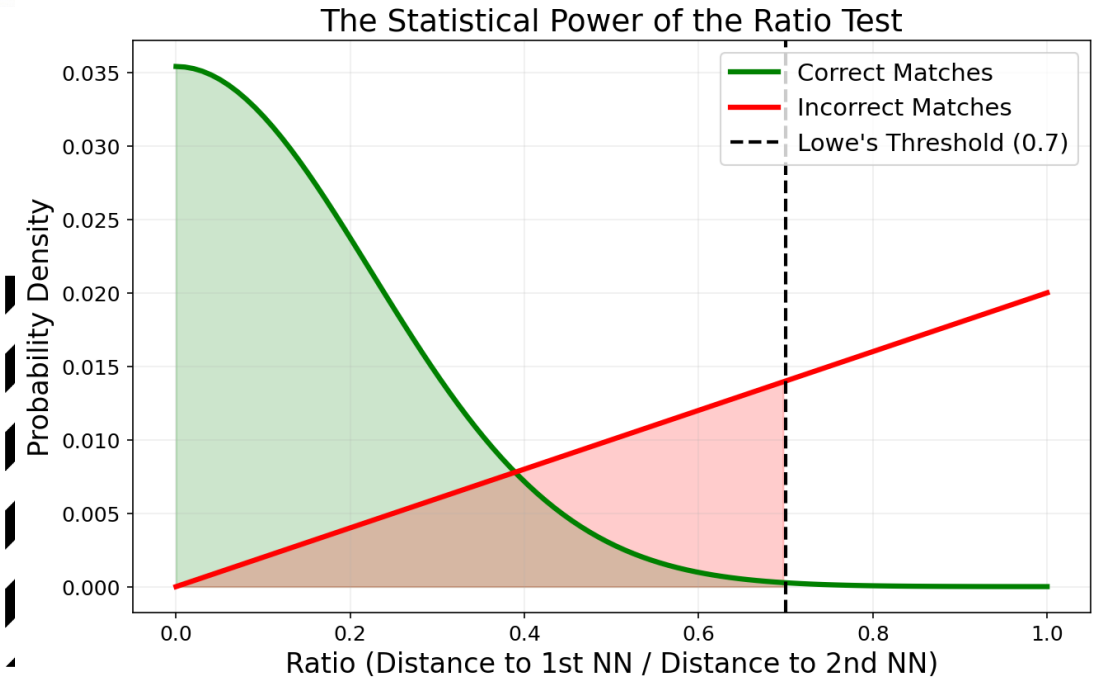
KNN + Lowe Ratio Test

Find **k=2 nearest neighbours**, use the distance **ratio** as a confidence score:

$$\frac{d_1}{d_2} < 0.75 \quad \Rightarrow \quad \text{keep}$$

- **Good match**: $d_1 \ll d_2$, ratio near 0 → keep
- **Repetitive texture**: $d_1 \approx d_2$, ratio near 1 → discard
- **No correspondence**: both large and similar → discard





Ratio Test: OpenCV Code

```
bf = cv2.BFMatcher()
knn_matches = bf.knnMatch(des1, des2, k=2)

good = []
for m, n in knn_matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)
```



Ratio Test Result



After Ratio Test: false matches drop dramatically, remaining matches are far more reliable



RANSAC

Random Sample Consensus



Why Do We Still Need RANSAC?

Ratio Test only checks each match **individually** — not **geometric consistency**:

Remaining false matches	Reason
Repetitive texture	Descriptors too similar; Ratio Test can't distinguish
Locally similar patches	Patches look alike but are not the same 3D point

Key insight: all correct matches must satisfy the **same global geometric transform** H

Homography is a matrix describing the **projective map** between two images for a **planar scene** (or pure camera rotation)

It has **8 degrees of freedom** (9 entries minus scale). Each point pair gives 2 equations, so **at least 4 pairs** are needed to solve for .

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad H \in \mathbb{R}^{3\times 3}, \ \text{rank} = 3$$



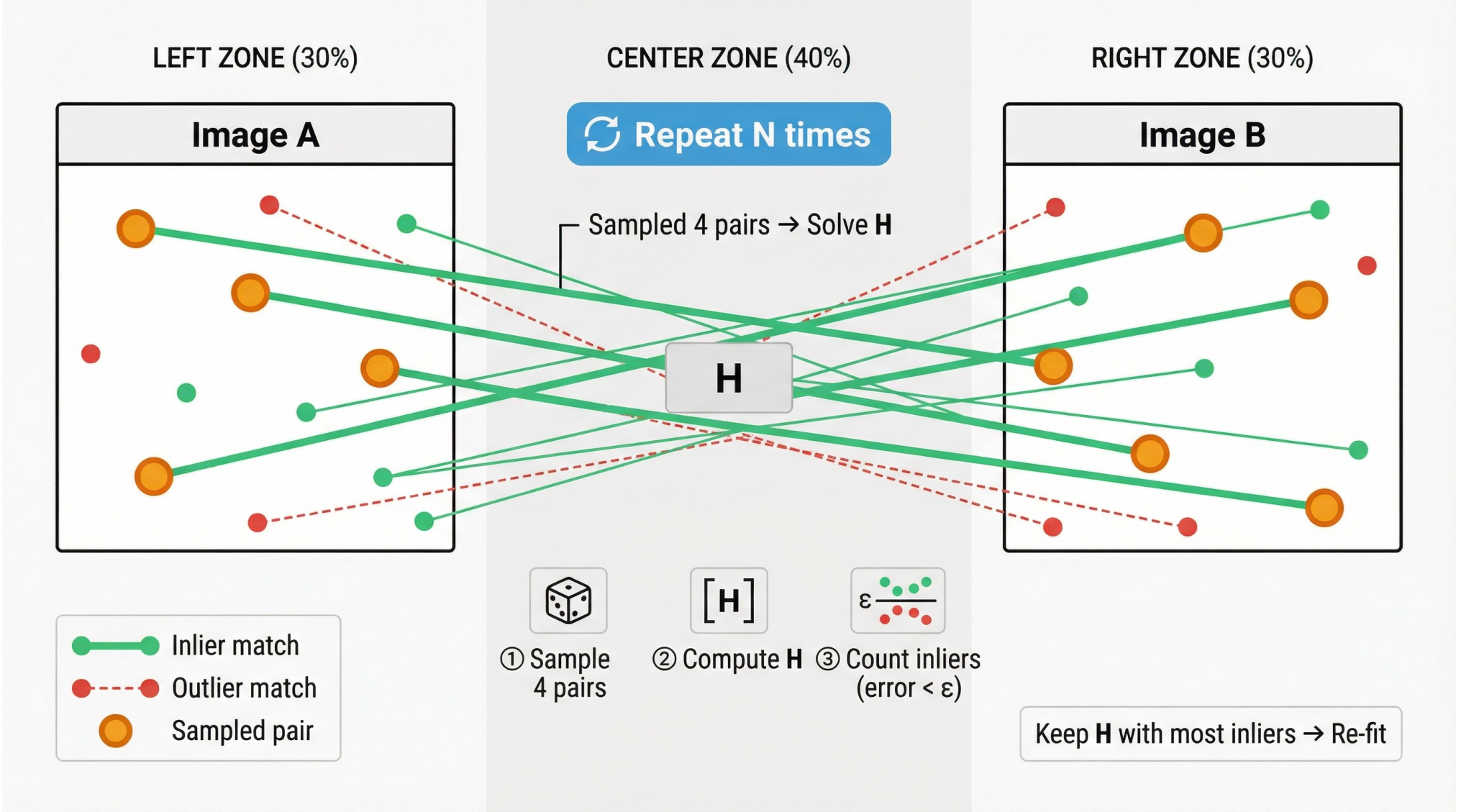
RANSAC Algorithm

```
Repeat N times:
  1. Randomly sample 4 non-collinear point pairs
  2. Solve for candidate homography H (unique solution)
  3. Compute reprojection error for all matches
     error < threshold (e.g. 5 px) → inlier
  4. Record inlier count

After N iterations:
  5. Keep H with the most inliers
  6. Re-fit H using all inliers (refinement)
```



RANSAC Algorithm — Visual Overview



Why does random sampling work?

If the inlier ratio is w and we sample $n = 4$ points, the probability that all 4 are inliers is w^4. After N trials, the probability of "at least one all-inlier sample" is:

$$P = 1 - (1 - w^n)^N$$

For $P = 0.99$ (99% confidence), required iterations:

$$N = \frac{\log(1 - P)}{\log(1 - w^n)}$$

Inlier ratio w	**Min iterations** N **(**$n = 4$**)**
50%	72
30%	468
20%	2498

Note: The table uses practical values (≈98% confidence); exact formula gives slightly different numbers. As outliers increase, N grows quickly. Even 2498 iterations take only milliseconds, so RANSAC is very efficient.

With 80% wrong matches, ~2500 iterations are enough to find a good H with 99% probability.



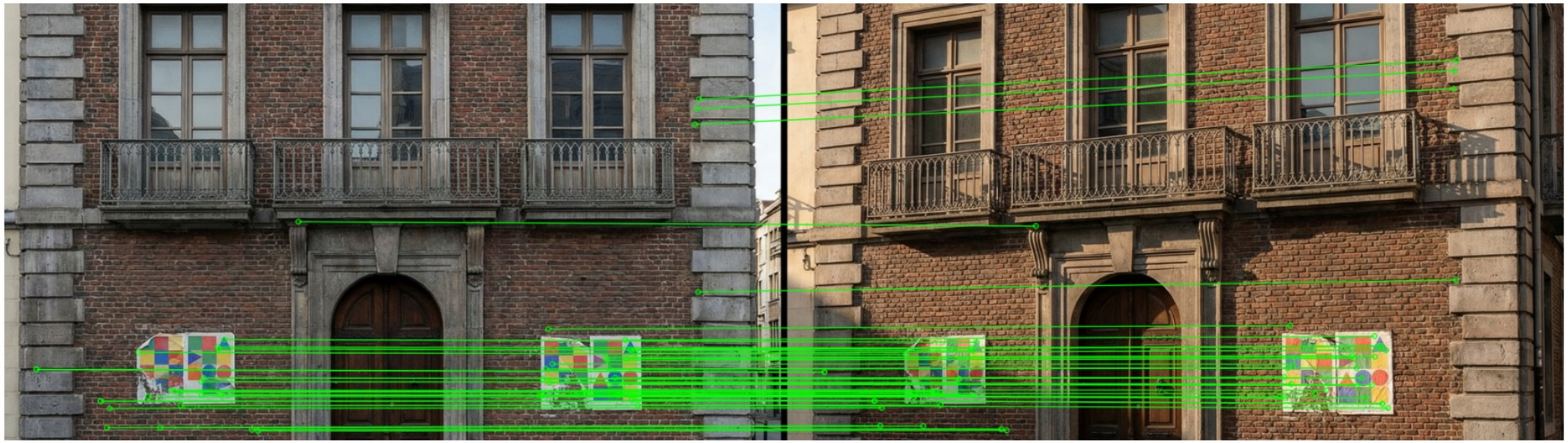
Homography Matrix

Projective transform between two views of a planar scene:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- 8 degrees of freedom (9 elements − scale ambiguity)

Final: SIFT + Ratio + RANSAC



RANSAC: OpenCV Code

```
src_pts = np.float32(
    [kp1[m.queryIdx].pt for m in good]
).reshape(-1, 1, 2)
dst_pts = np.float32(
    [kp2[m.trainIdx].pt for m in good]
).reshape(-1, 1, 2)

H, mask = cv2.findHomography(
    src_pts, dst_pts, cv2.RANSAC, 5.0)
inliers = [good[i] for i in range(len(good)) if mask[i]]
```



Verification: Warp + Overlay

```
h, w = img2.shape[:2]
img1_warped = cv2.warpPerspective(img1, H, (w, h))
overlay = cv2.addWeighted(img1_warped, 0.5, img2, 0.5, 0)
```

Warp image A into image B's coordinate frame and blend — aligned means H is correct



Typical Failure Cases

- **Textureless regions**: white walls, sky → no corners found
- **Repetitive texture**: grid windows, brick → descriptors not distinctive
- **Large deformation**: clothing, facial expressions → Homography breaks
- **Heavy occlusion**: only a small region remains visible
- **Extreme lighting**: blown-out highlights or deep shadows

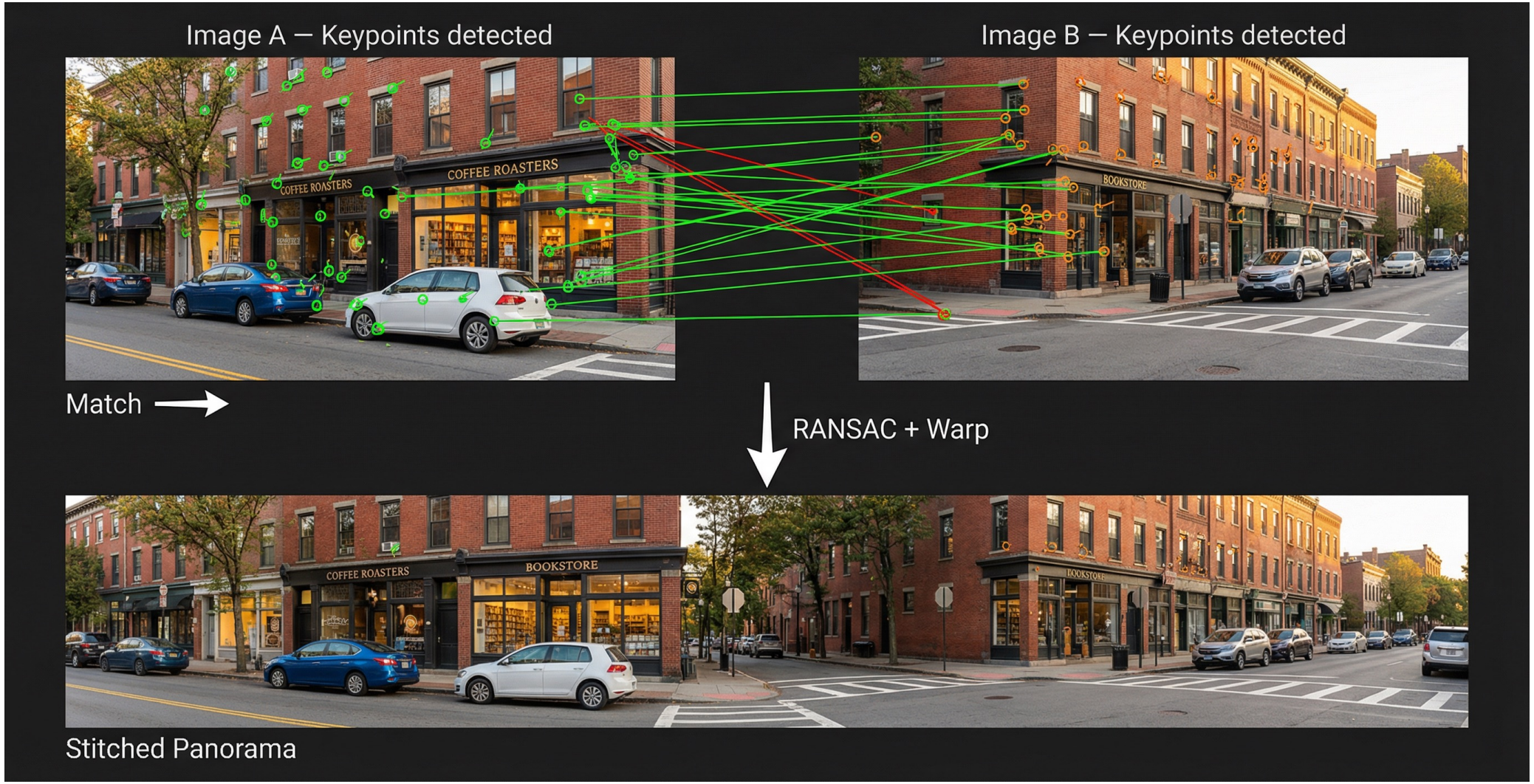
ORB (Oriented FAST + Rotated BRIEF), proposed in 2011, is an open alternative to SIFT—**~100× faster, no patent**.



Applications



Stitch Panorama



Localization



Large Scale: Visual Search

Bag of Visual Words (BoW)

1. **Vocabulary Building**:
   - Extract millions of SIFT points; run **K-Means Clustering** to find K (e.g., 10k) centers.
   - Each center is a **Visual Word**. The set is the **Codebook**.
2. **Encoding**:
   - **Quantization**: Map each 128-D SIFT descriptor to the closest Visual Word ID.
   - **Histogram**: Count word occurrences.
   - **Fixed-Length Vector**: Every image becomes a K**-dim vector** for fast comparison.
3. **Retrieval**:
   - Compare histograms using **Inverted Index** to skip database scans.

